

05 — LTL Properties

Safety, Liveness, and Response Patterns

Mariano Cerrutti

Mununu Tutorial

LTL Operators

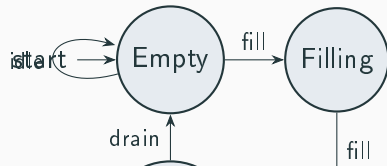
Syntax	Operator	Meaning
G	G (Globally)	φ holds in <i>every</i> future state
F	F (Finally)	φ holds in <i>some</i> future state
X	X (Next)	φ holds in the <i>next</i> state
U	U (Until)	ψ holds eventually; φ holds until then
W	Weak Until	Like U, but ψ need not occur
R	Release	Dual of U: ψ holds until $\varphi \wedge \psi$
->	Implies	Standard logical implication

In Mununu, LTL formulas are written with `body = 1t1 ...`; and automatically translated to mu-calculus for evaluation.

The Water Tank Automaton

```
1  states {  
2      state Empty initial;  
3      state Filling;  
4      state Full;  
5      state Draining;  
6  }
```

```
7  
8  transitions {  
9      // Start filling when  
10     empty  
11     transition Empty ->  
12         Filling on label  
            fill;  
13     // Tank is now full  
14     transition Filling ->  
            Full on label  
            fill;  
15     // Tank is empty again  
16     transition Full ->  
            Draining on label  
            drain;  
17     transition Draining ->  
            Empty on label  
            drain;  
18 }
```



Three Property Types

```
1  mu_formulas {
2      // === LTL Properties ===
3
4      // Safety: the tank is always in a valid state.
5      // G(phi) means phi holds in every reachable state.
6      formula ltl_safety {
7          over Tank;
8          body = ltl G (Empty || Filling || Full || Draining);
9      }
10
11     // Liveness: if the tank starts filling, it eventually becomes full.
12     // G(p -> F(q)) is the "response" pattern.
13     formula ltl_liveness {
14         over Tank;
15         body = ltl G (Filling -> F Full);
16     }
17
18     // GP(1) response: if the tank is full infinitely often
```

LTL $\leftrightarrow$ Mu-Calculus Equivalence

Pattern	LTL	Mu-calculus
Safety (invariant)	$G\ safe$	$\nu X. \Box X$
Reachability	$F\ target$	$\mu X. target \vee \Diamond X$
Infinitely often	$GF\ p$	$\nu X. (\mu Y. p \vee \Diamond Y) \wedge \Box X$

Equivalent mu-calculus formulas from the same file:

```
1 // === Equivalent mu-calculus formulas (for comparison) ===
2
3 // Safety in mu-calculus: nu X. ([] X) --greatest fixpoint invariant
4 formula mu_safety {
5     over Tank;
6     body = nu X. ([] X);
7 }
8
```

Game-Theoretic Reachability

Can the controller *force* the system to reach **Full**, despite the environment choosing *idle*?

```
1
2 // Controllable reachability: can the controller force reaching Full
3 // despite the environment's ability to choose 'idle'?
4 //
5 // [ (ctrl = controllable) ] X means:
6 // - ALL uncontrollable transitions must lead to X (environment can't
   //   escape)
7 // - At least ONE controllable transition must lead to X (controller has a
   //   strategy)
8 //
9 // With 'idle' uncontrollable: FAILS for Empty and Draining (2/4)
10 // because the environment can loop idle forever at Empty.
11 // If idle were controllable: SUCCEEDS for all states (4/4)
12 // because the controller would simply choose fill instead of idle.
13 formula ctrl_reachable_full {
```

Syntax: LTL vs. Mu-Calculus

Both formula styles use the same `formula` block:

LTL syntax:

```
1 formula ltl_safety {  
2     over Tank;  
3     body = ltl G (Empty || Filling  
4               || Full || Draining);  
5 }
```

Mu-calculus syntax:

```
1 formula mu_safety {  
2     over Tank;  
3     body = nu X. ([] X);  
4 }
```

- Use `body = ltl ...`; for temporal logic patterns.
- Use `body = ...`; (no `ltl` keyword) for mu-calculus.
- Mununu translates LTL to mu-calculus internally before evaluation.

References: Pnueli (1977) “The Temporal Logic of Programs”; Piterman, Pnueli & Sa’ar (2006) “Synthesis of Reactive(1) Designs”.